Graph Engineering

Designing the topology of multi-agent systems, and the graphs they remember with

Agent graphs · Production · Knowledge graphs · Shared memory

Part 1: Agent graphs: foundations

Graph vs. loop · Primitives · State and reducers · Durable execution

Part 2: Agent graphs in production

Topology patterns · When not to build · Failure modes · Governance

Part 3: Knowledge graphs

Fundamentals and Cypher · Building KGs with LLMs · KG-RAG and text-to-Cypher

Part 4: Where the graphs meet

KG as shared agent memory · Learning on graphs

Notes: ai-engineering/05-graph-engineering

PART 1

Agent graphs: foundations

Graph vs. loop · Primitives · State and reducers · Durable execution

The definition

Workflows are graphs; agents are loops inside them

Each layer fails differently

Five signals that justify a node

The 30-second decision tree

Two things called "graph"

Spend agency per node

The minimal graph program

The highest-yield first graph: generate-then-verify

Typed state with a reducer

Parallel writes without a reducer fail

Dynamic fan-out: the Send API

Checkpoints make a graph pausable

Static breakpoints vs. interrupt()

Gate the edges where consequence concentrates

The definition

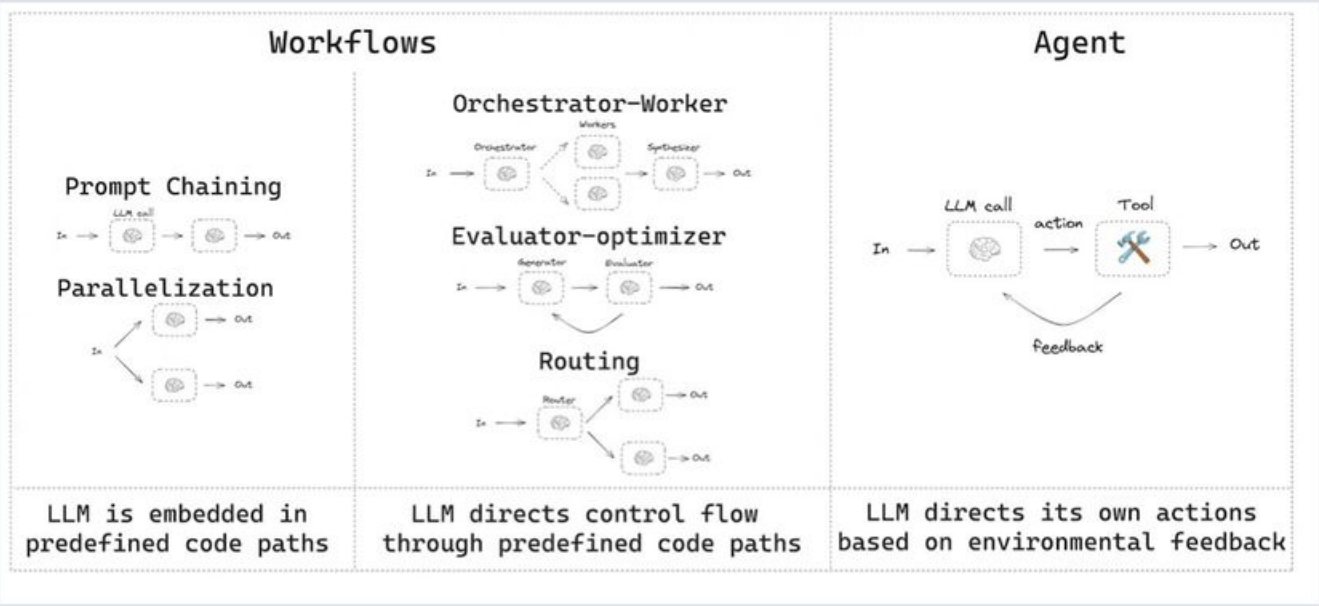
> **Graph engineering is designing which nodes exist, which transitions are permitted, and what state they carry.**

"A loop is just a directed, cyclic graph." (LangChain, 3 years of LangGraph)

Source: notes/01-graph-vs-loop.md

Workflows are graphs; agents are loops inside them

Anthropic, Building Effective Agents (2024)



LangGraph docs, "Workflows and agents", after Anthropic (2024)

- **Workflow:** predefined code paths
- **Agent:** the model directs its own process

- A graph writes the workflow down
- Most production graphs: fixed shapes with **one or two agent nodes** inside

Source: notes/01-graph-vs-loop.md

Each layer fails differently

Loop

- Doesn't converge
- Spins or quits early
- Games the scorekeeper

Fix the loop first

Graph

- Mis-routes
- Deadlocks at a join
- Double-writes shared state
- Loses accountability for a decision

Production agents need cycles: not DAGs

A graph of unreliable loops inherits every loop failure, then adds coordination failures.

Source: notes/01-graph-vs-loop.md

Five signals that justify a node

Start with a loop. Add a node only when one of these forces it.

Signal	Why the loop fails
Distinct specialties	One agent carries each role's baggage into the next
Parallel fan-out + join	A loop is sequential; wall-clock cost is unnecessary
Per-step model or tool	One agent enforces one model and one tool policy
Auditable control flow	Emergent loop paths are hard to reconstruct
Overloaded verifier	A dedicated reviewer removes criteria confusion

> **Escalation is a cost you justify, not a maturity level you graduate to.**

Source: notes/01-graph-vs-loop.md

The 30-second decision tree

ask in order

1. **Specialties?** — Does the work split into distinct specialties with different needs?
2. **Parallelism?** — Do you need true parallelism: fan out, then join?
3. **Variation?** — Do steps need different models or tools, or auditable branching?
4. **Verifier?** — Is one verifier failing because it judges too many criteria at once?

★ **No to all — Strengthen the verifier and ship the loop.**

> **The most common "we need a graph" is really an overloaded verifier.**

Source: notes/01-graph-vs-loop.md

Two things called "graph"

	Agent graph (Parts 1–2)	Knowledge graph (Part 3)
Nodes are	Units of computation	Entities
Edges are	Permitted transitions	Typed relationships
The graph is	Execution topology	Data structure for retrieval
Runtime concern	Routing, state, cost	Traversal, entity resolution

> **They meet in Part 4, where the knowledge graph is the shared state agents coordinate through.**

Source: notes/01-graph-vs-loop.md

Spend agency per node

Node kind	Purpose	Determinism	Cost
Deterministic function	Parsing, validation, API calls	Total	Compute only
Router	Classify and dispatch	Medium	One call at most
Single LLM call	Summarise, extract, draft	Medium	One call
Agent	Open-ended reasoning with a tool loop	Low	Many, variable
Human checkpoint	Approval before a consequential act	External	Latency

> **Code where you can, one LLM call where you must, an agent only where the path is unknown.**

Source: notes/02-graph-primitives.md

The minimal graph program

PYTHON

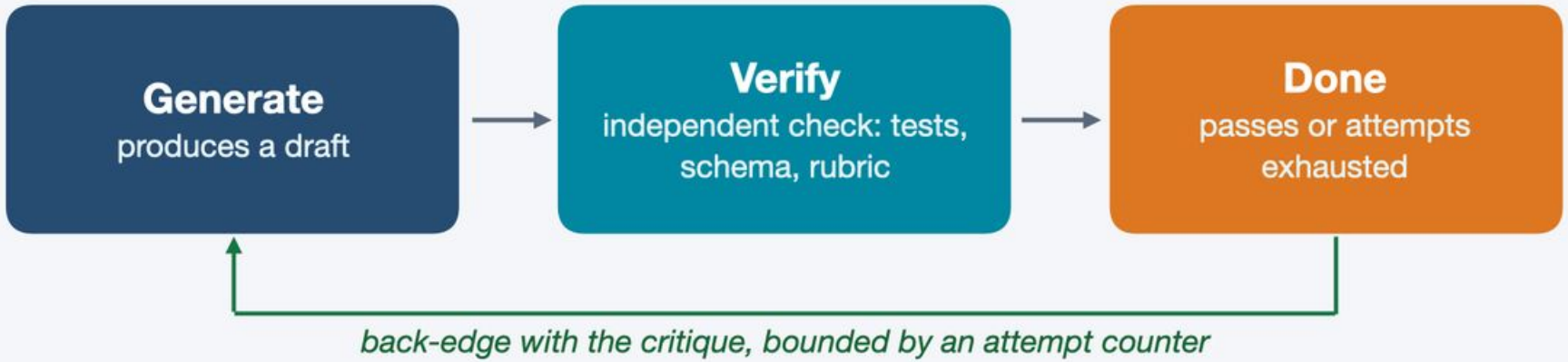
```python
builder = StateGraph(MessagesState)
builder.add_node("node_a", node_a)
builder.add_node("node_b", node_b)
builder.add_edge(START, "node_a")
builder.add_conditional_edges("node_a", router)
builder.add_edge("node_b", "node_a")  # the cycle
graph = builder.compile()

def router(state) -> str:
   # ordinary code over state
   return "node_b" if needs_more(state) else END
```

- **Nodes:** read state, return only changed keys
- **Router:** plain code returning the next node's name
- **Conditional edges** give branching, error paths, gates, early exit, cycles
- Compile, then invoke with initial state

Source: notes/02-graph-primitives.md

The highest-yield first graph: generate-then-verify



- The verifier sees the task and the output, **never the generator's reasoning**
- The back-edge carries the critique into the next attempt

Typed state with a reducer

PYTHON

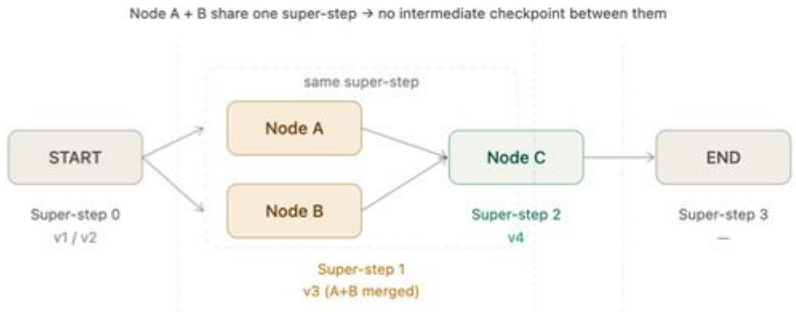
```python
class ReviewState(TypedDict):
    task: str              # set once, read by all
    draft: str             # one writer: generator
    findings: Annotated[list[str], operator.add]
                           # many writers: reducer
    attempts: int          # the bound on the cycle
```

- Every key has **one writer or a reducer**
- `operator.add` concatenates parallel writes
- Custom reducers can dedupe
- `attempts` lives in state so the router can bound the cycle

Source: notes/03-state-and-reducers.md

Parallel writes without a reducer fail

Case 2 — Parallel execution: flow diagram



Case 2 — Parallel execution: checkpoint table

Checkpoint	Trigger (super-step boundary)	State snapshot	Next node(s)
v1	Graph initialized	`{}` (empty)	`__start__`
v2	User input processed	`{"foo": "", "bar": []}`	`node_a`, `node_b`
v3	Node A *and* Node B both complete — reducer merges results	`{"foo": "b", "bar": ["a", "b"]}`	`node_c`
v4	Node C completes	`{"foo": "c", "bar": ["a", "b", "c"]}`	`END`

LangGraph docs (LangChain), via sources/langgraph_extended.pptx

- Branches A and B run in **one super-step**
- No checkpoint between them
- Both write one key with no reducer → **`InvalidUpdateError`**
- Outside LangGraph the same race silently loses work

> **Every shared key has one writer or a reducer.**

Source: notes/03-state-and-reducers.md

Dynamic fan-out: the Send API

PYTHON

```python
def fan_out(state) -> list[Send]:
    return [Send("review_file",
                 {"path": p, "task": state["task"]})
            for p in state["changed_files"]]

builder.add_conditional_edges(
    "plan", fan_out, ["review_file"])
builder.add_edge("review_file", "synthesize")
```

- One worker per item, decided **at runtime**
- Each `Send` carries its own private input
- Results merge through the reducer
- **Cap N**: unbounded fan-out is a cost bug

> **Does the next step read the previous step's output? If not, fan out.**

Source: notes/03-state-and-reducers.md

Checkpoints make a graph pausable

PYTHON

```python
graph = builder.compile(checkpointer=saver)
config = {"configurable": {"thread_id": "review-42"}}

def publish(state):
    decision = interrupt({"draft": state["draft"]})
    if decision != "approve":
        return {"status": "rejected"}
    send_to_cms(state["draft"])   # after the interrupt
    return {"status": "published"}

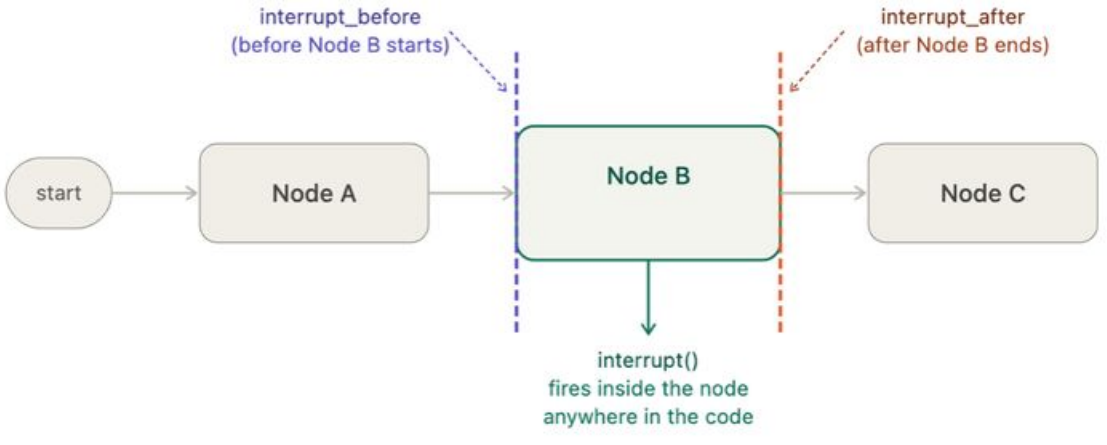
graph.invoke({"task": "..."}, config)       # pauses
graph.invoke(Command(resume="approve"), config)
```

- **Checkpoint** after every super-step
- **Thread** = `thread_id`; pick it deliberately
- Human-in-the-loop, crash resume, time travel
- Gotcha: the node **reruns from line 1** on resume

> **If a run can outlive its process, compile with a persistent checkpointer.**

Source: notes/04-durable-execution-and-interrupts.md

Static breakpoints vs. interrupt()



LangGraph docs, human-in-the-loop (LangChain)

- `interrupt_before/after`: node boundaries, for **debugging**
- `interrupt()`: anywhere inside a node, for **production**
- The node passes the human exactly what they need
- Put it inside a **tool** and the gate travels with the tool

Source: notes/04-durable-execution-and-interrupts.md

Gate the edges where consequence concentrates

"Human approval checkpoints ... at exactly the edges where consequence concentrates." (TrueFoundry)

Gate before	Why
Irreversible actions: send, pay, delete, deploy	Can't be undone, so check first
Crossing a trust boundary: external API, other tenant	Someone else bears the cost
Low-confidence router decisions	Cheap for a human, expensive to mis-route
After the plan, before the build (n8n)	The plan is the cheapest thing to correct

> **A gate on every node trains people to click approve without reading.**

Source: notes/04-durable-execution-and-interrupts.md

PART 2

Agent graphs in production

Topology patterns · When not to build · Failure modes · Governance

Five topologies that recur

Subgraphs: share keys, or map at the boundary

Shared vs. isolated context

A second agent helps only if it adds information

Fan-out is expensive

Loop or graph?

Mechanical failures come from the wiring

Swarms fail in ways single agents don't

Control failures and the four principles

Operate the graph, don't just demo it

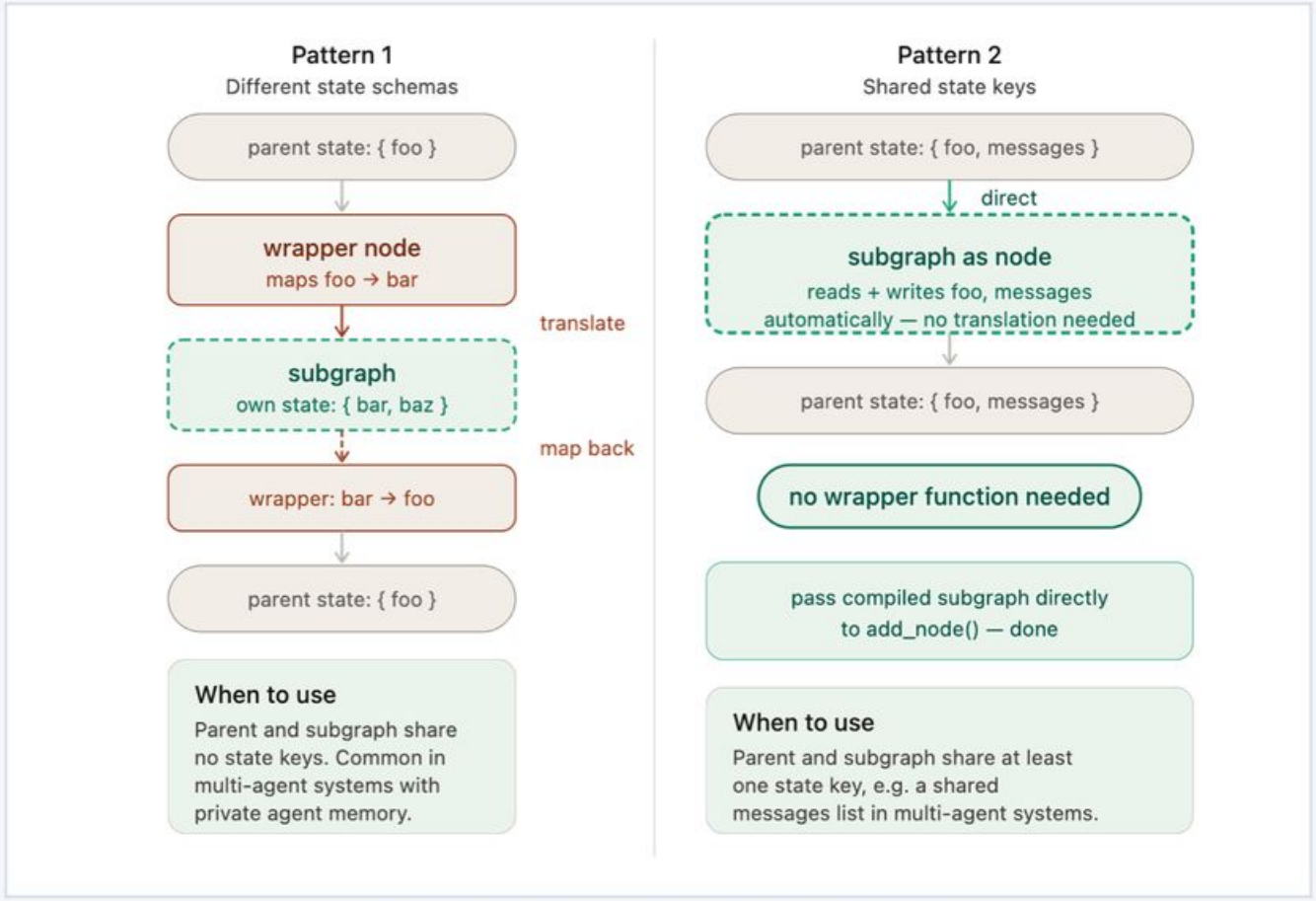
Case study: n8n's AI workflow builder

Five topologies that recur

Pattern	Shape	Use when	Fails by
Generate-then-verify	produce → check → back-edge	Produce and check split cleanly	Checker sees producer's reasoning
Supervisor	router → specialists → router	Work arrives in several kinds	Mis-routing; bottleneck
Fan-out / fan-in	split → N workers → merge	Many independent items	Missing reducer; correlated errors
Hierarchical	subgraph as one node	Reused or team-owned sub-topology	Failures hidden inside
Handoff	status key → edge reads it	Agents shouldn't know each other	Status values drift

Source: notes/05-topology-patterns.md

Subgraphs: share keys, or map at the boundary

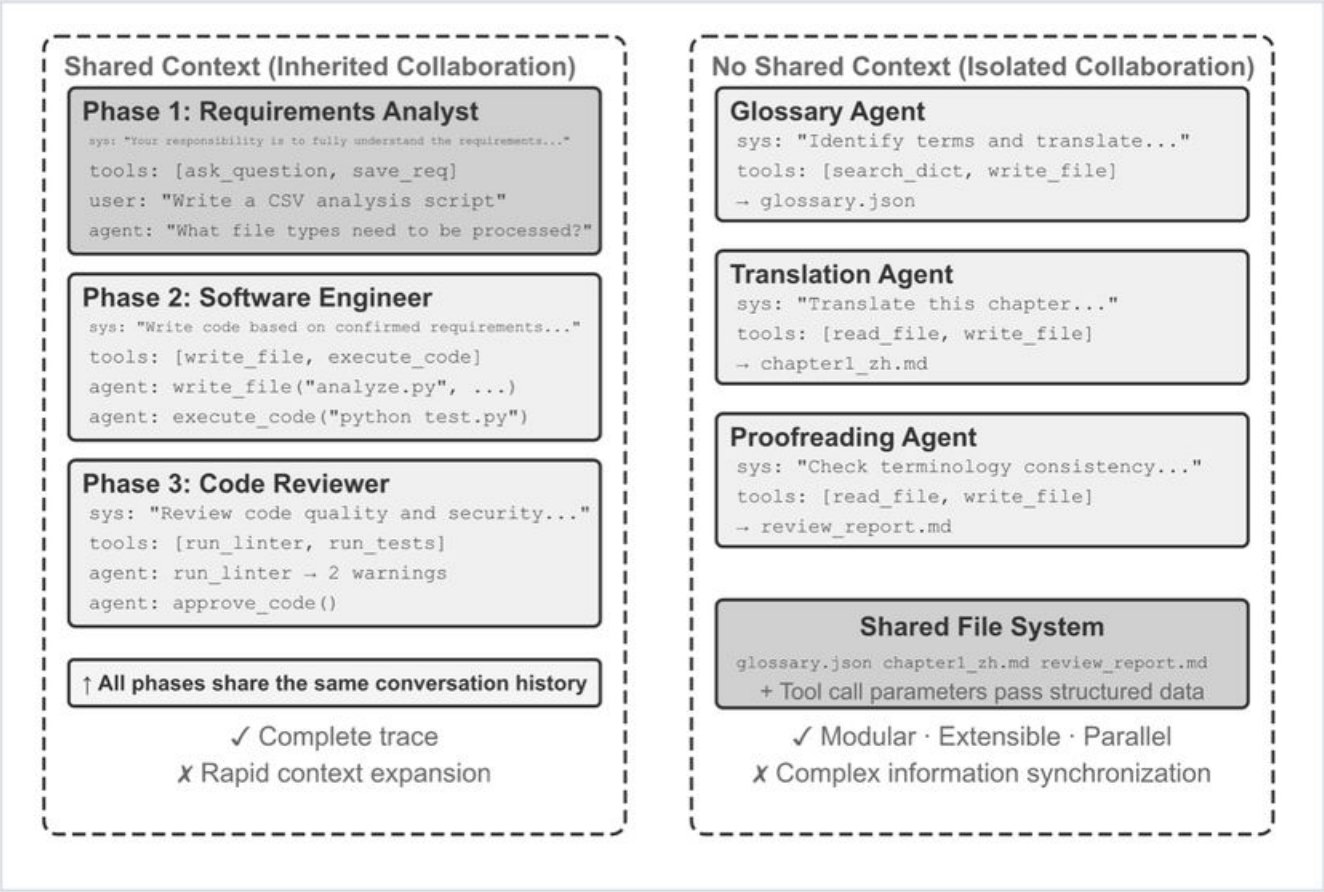


LangGraph docs, subgraphs (LangChain)

- **Shared keys:** add the compiled subgraph directly
- **Different schemas:** call it inside a wrapper node
- The wrapper maps parent → child input and child output → parent update

Source: notes/05-topology-patterns.md

Shared vs. isolated context



Li, AI Agents in Depth, ch. 10, Fig. 10-1 (Apache-2.0)

- **Shared:** one trajectory, like threads; nothing lost, everything accumulates
- **Isolated:** own window, like processes; only what is passed explicitly
- Isolation keeps a verifier independent

Source: notes/05-topology-patterns.md

A second agent helps only if it adds information

Extra agent brings	Helps?	Evidence
Debate among copies of one model	No	Matches a single agent given the same compute (Tran & Kiela)
Execution, tool or visual feedback	**Yes**	WebGen-Agent screenshots: Claude 3.5 Sonnet **26.4%** → **51.9%**

> **Recombining the same information can't add any. New signals can.**

Source: notes/05-topology-patterns.md

Fan-out is expensive

Anthropic multi-agent research system

Tokens

Agents use **~4×** a chat's tokens; multi-agent systems **~15×**

Why it works

Token usage alone explained most of the performance variance

Break-even

Fan-out ~**3×** tokens per cycle; break-even near a **50%** pass rate (flowtivity)

Poor fit

Shared context everywhere, many inter-agent dependencies, or work you can't name in advance

Fan-out pays only for high-value work that parallelises well. Measure break-even.

Source: notes/06-when-not-to-build-a-graph.md

Loop or graph?

Situation	Use
Single agent, tool-augmented reasoning	Loop
Open-ended research with unknowable steps	Harness + loop, **not** a graph
Simple task, low per-cycle pass rate	Loop: fan-out cost isn't recovered
Verifier failing on several criteria	Split the verifier first
Several agents with conditional handoffs	Graph
3+ independent verification steps	Graph (parallel review)
Resume or human approval mid-run	Graph (checkpointer + interrupts)

Source: notes/06-when-not-to-build-a-graph.md

Mechanical failures come from the wiring

Failure	Detect	Fix
Infinite cycle	`GraphRecursionError` (`recursion_limit`, default **25**)	Attempt counter in state; per-node bound
Deadlock	Run idle at a join	Explicit join inputs; error nodes still write the key
Race on shared state	`InvalidUpdateError`, or silent loss	A reducer or one writer per key
Unbounded fan-out	Token spend spikes; rate limits	Cap N; budget per run and per node
Mis-routing	Route distribution in traces	Test the router as a classifier
File-space collision	Diffs match no single worker	One worktree per worker

Source: notes/07-failure-modes-at-scale.md

Swarms fail in ways single agents don't

Correlated errors

Shared model, prompt and index: N agreeing workers ≠ N independent ones

Verifier isolation

A checker that sees the reasoning agrees with it

Byzantine

Agents fail by being confidently wrong

Cascading amplification

One bad output becomes every downstream input

MAST

14 failure modes in 3 groups, κ = **0.88**

Structure beats wording

Better prompts and roles fixed ChatDev by only **+15.6%**

Most multi-agent failures are structural. Fix topology and verification, not wording.

Source: notes/07-failure-modes-at-scale.md

Control failures and the four principles

Many feedback loops interacting fail like coupled controllers.

Failure	Principle	How
Goodhart	Paired metrics	Every optimised metric gets a counter-metric and an anchor
Upward blindness	Owned references	Targets are owned by slower loops
Inter-loop conflict	Separated cadences	Fast loops escalate to slow ones instead of overriding
Measurement decay	Frozen nodes	Some measurements are deliberately not tunable

> **Keep at least one anchor the graph can't edit.**

Source: notes/07-failure-modes-at-scale.md

Operate the graph, don't just demo it

Identity

- `graph_id`: topology + version
- `run_id` / `thread_id`: one execution
- `node_id`: per-node cost and errors

Budgets

- **Per node:** one node looping
- **Per run:** a whole run going wrong
- **Per tenant:** one customer draining the pool

Checklist

- Resolved caller identity
- Record the actual runtime graph
- Join traces with cost and policy
- Model changes behind a rollback alias

Source: notes/08-governance-and-production.md

Case study: n8n's AI workflow builder

Supervisor + 5 subgraphs

Node	Role	Bound
Supervisor	Routes each message to a subgraph	—
Discovery	Explores the node registry	**50** iterations
Planner	Builds a structured plan; **pauses for approval**	—
Builder	Executes tool calls	**100** per session
Responder	Renders the user-facing reply	—
Parameter Updater	Surgical edits to node parameters	—

Published budgets: 200K window, compact at 150K, 30K max workflow JSON.

Source: notes/08-governance-and-production.md

PART 3

Knowledge graphs

Fundamentals and Cypher · Building KGs with LLMs · KG-RAG and text-to-Cypher

A knowledge graph: entities and typed relationships

Cypher draws patterns

Errors compound per hop

From text to a graph you can trust

Keep mentions; never merge at extraction time

Extract, enhance, expand

Expand: fuse structured data with MERGE

Where vector RAG breaks

What GraphRAG actually showed

Five retrieval modes

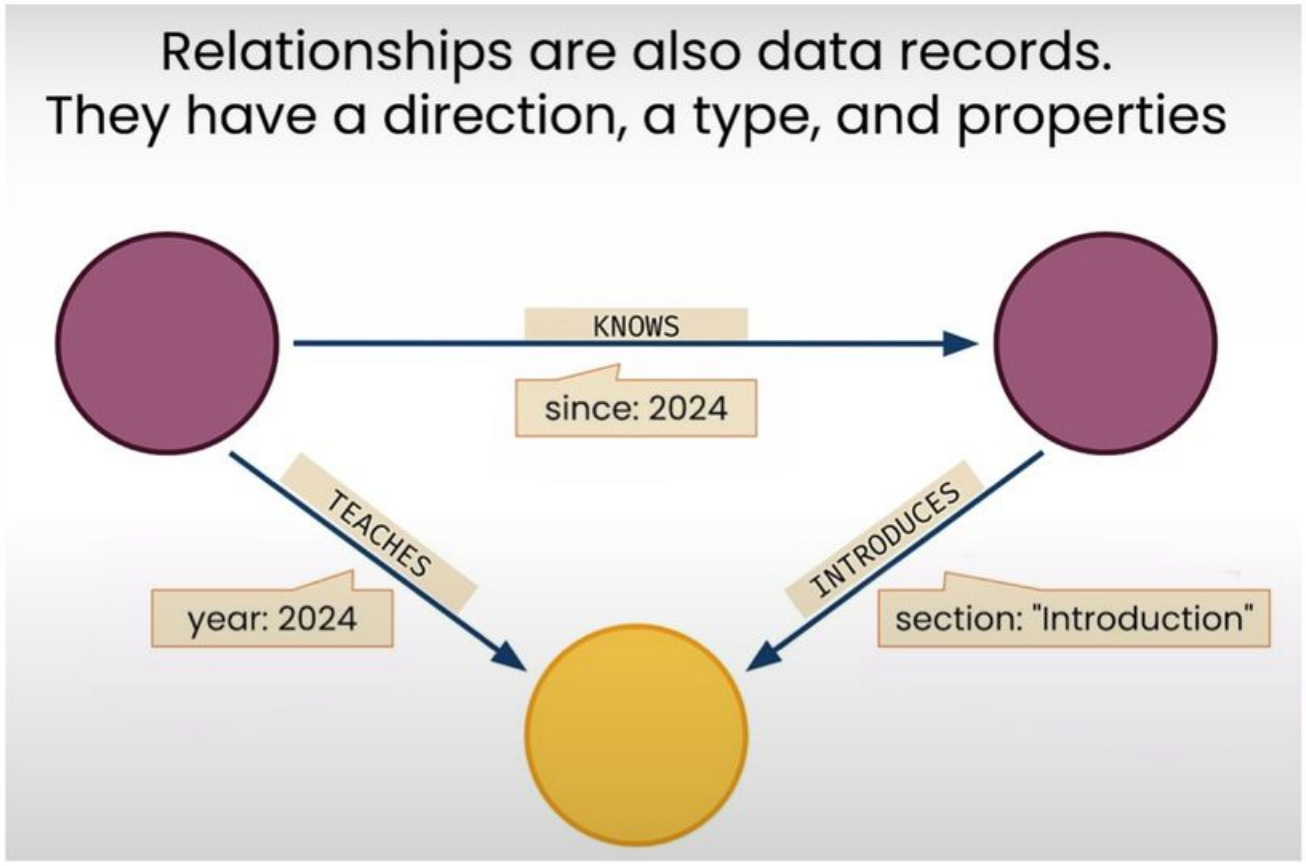
Hybrid in practice: a retrieval_query

Text-to-Cypher: think like the expert

The failure to expect, and its one-example fix

The QA agent as a LangGraph graph

A knowledge graph: entities and typed relationships



Knowledge Graphs for RAG, DeepLearning.AI × Neo4j (Kollegger, 2024)

- **Nodes** carry labels and properties
- **Relationships** are typed, directed and carry properties too
- **LPG** (Neo4j): properties on edges, Cypher
- **RDF:** triples, SPARQL, ontologies

Source: notes/09-kg-fundamentals-and-cypher.md

Cypher draws patterns

CYPHER

```
// co-actors of a person
MATCH (p:Person {name: $name})
      -[:ACTED_IN]->(m:Movie)
      <-[:ACTED_IN]-(co:Person)
RETURN co.name, collect(m.title) AS movies

// idempotent load: constraint + MERGE
CREATE CONSTRAINT unique_chunk IF NOT EXISTS
 FOR (c:Chunk) REQUIRE c.chunkId IS UNIQUE;
MERGE (c:Chunk {chunkId: $id})
 ON CREATE SET c.text = $text
```

- `()` nodes, `-[]->` relationships
- **MERGE** = match or create
- A uniqueness constraint makes reloads safe
- Python: `Neo4jGraph.query(cypher, params=...)` from `langchain-neo4j`

Source: notes/09-kg-fundamentals-and-cypher.md

Errors compound per hop

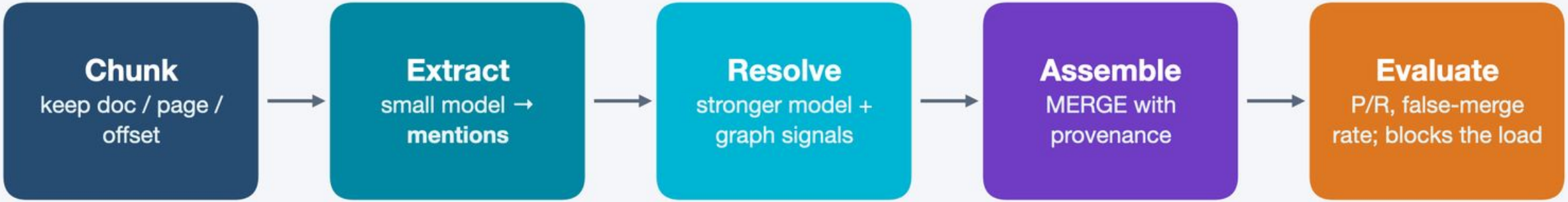
If each hop is right with probability p, an n-hop answer is right with p^n.

Per-hop accuracy	2 hops	3 hops	5 hops
95%	90%	86%	77%
85%	72%	61%	**44%**
70%	49%	34%	17%

> **Entity resolution is the load-bearing step. One false merge corrupts every path through it.**

Source: notes/10-building-kgs-with-llms.md

From text to a graph you can trust



- Normalise (deterministic) sits between extract and resolve
- Same shape as Anthropic's KG cookbook: Haiku extracts, Sonnet resolves
- Use the model only where judgement is needed

Source: notes/10-building-kgs-with-llms.md

Keep mentions; never merge at extraction time

CYPHER

```
// raw mentions
(:Document)-[:HAS_PAGE]->(:Page)
 -[:MENTIONS]->(:Entity)
// resolution evidence
(:Entity)-[:META_SIMILAR]->(:Entity)
// the KG proper
(:Entity)-[:RESOLVES_TO]->(:Canonical)

// graph-based ER: weakly connected components
// over META_SIMILAR → one canonical each
```

- The metagraph keeps resolution **reversible**
- Weight **false merges** above missed merges
- A high compression ratio rewards over-merging

Source: notes/10-building-kgs-with-llms.md

Extract, enhance, expand

	Chunked Text Nodes	Form 10K Nodes	Companies	Management Firms
Source	Form 10K json	(:Chunk)	Form 13 CSV	Form 13 CSV
1. Extract	`(:Chunk)`	`(:Form)`	`(:Company)`	`(:Manager)`
2. Enhance	Vector embedding of text	Unique chunk IDs	Unique company cusip6	Full-text index of names
3. **Expand**	`(Chunk) -[NEXT]→ (Chunk)`	`(Chunk) -[PART_OF]→ (Form)`	`(Company) -[FILED]→ (Form)`	`(Manager) -[OWNS_STOCK_IN]→ (Company)`

Knowledge Graphs for RAG, DeepLearning.AI × Neo4j (Kollegger, 2024)

- **Extract:** 10-K text → `Chunk` nodes with embeddings
- **Enhance:** `NEXT` and `PART_OF` give chunks a document structure
- **Expand:** fuse structured Form 13 data: managers and holdings
- Start with a minimum viable graph

Expand: fuse structured data with MERGE

CYPHER

```
CREATE CONSTRAINT unique_manager IF NOT EXISTS
 FOR (m:Manager) REQUIRE m.managerCik IS UNIQUE;

MERGE (com:Company {cusip6: $cusip6})
MERGE (mgr:Manager {managerCik: $managerCik})
MERGE (mgr)-[o:OWNS_STOCK_IN {
   reportCalendarOrQuarter: $quarter}]->(com)
 ON CREATE SET o.value = $value,
               o.shares = $shares
```

- One edge **per quarter**: the key is on the relationship
- Constraints make the load idempotent
- A full-text index on manager names handles fuzzy lookup

Source: notes/10-building-kgs-with-llms.md

Where vector RAG breaks

Failure	What happens
Context fragmentation	A fact split across chunks needs every piece retrieved near the top
Embedding compression	One vector per chunk loses entity-level detail
Noise	Loosely related chunks dilute the answer
Misses	Aggregate questions ("main themes?") get partial answers

> **"Lauritsen and cyclotrons": top 3 scored 0.906 / 0.903 / 0.902, and only one mentioned Lauritsen.**

Source: notes/11-kg-rag-and-text-to-cypher.md

What GraphRAG actually showed

Edge et al., arXiv 2404.16130 (2024)

Comprehensiveness

Win rates **72–83%** (podcasts), **72–80%** (news)

Diversity

Win rates **75–82%** and **62–71%**

Directness

Vector RAG wins: more concise answers

The caveat

LLM judge, global sensemaking questions over ~1M tokens

Community summaries help whole-corpus questions. That is not "graphs beat vectors".

Source: notes/11-kg-rag-and-text-to-cypher.md

Five retrieval modes

Mode	Use when	Fails when
Vector	Local lookups: "the passage about X"	Multi-hop, aggregates, exact entities
Canned traversal	Known question shapes; production paths	A shape you didn't template
Text-to-Cypher	Open-ended questions over a well-named schema	Large or badly named schema; no validator
Hybrid	Vector entry points + graph expansion	Wrong entry points make expansion worse
Global (communities)	Themes across everything	Specific lookups; high indexing cost

> **Default order: vector, then canned traversals, then text-to-Cypher for the long tail, with a validator.**

Source: notes/11-kg-rag-and-text-to-cypher.md

Hybrid in practice: a retrieval_query

CYPHER

```
// node and score come from the vector index
MATCH window = (:Chunk)-[:NEXT*0..1]->(node)
               -[:NEXT*0..1]->(:Chunk)
WITH node, score, window
 ORDER BY length(window) DESC LIMIT 1
WITH nodes(window) AS chunks, node, score
UNWIND chunks AS c
WITH collect(c.text) AS texts, node, score
RETURN apoc.text.join(texts, " \n ") AS text,
       score, node {.source} AS metadata
```

- `Neo4jVector` runs the search, then this Cypher
- Widens each hit to its `NEXT` neighbours
- L6 walks on to company and managers
- The windowed answer also named **ONTAP**

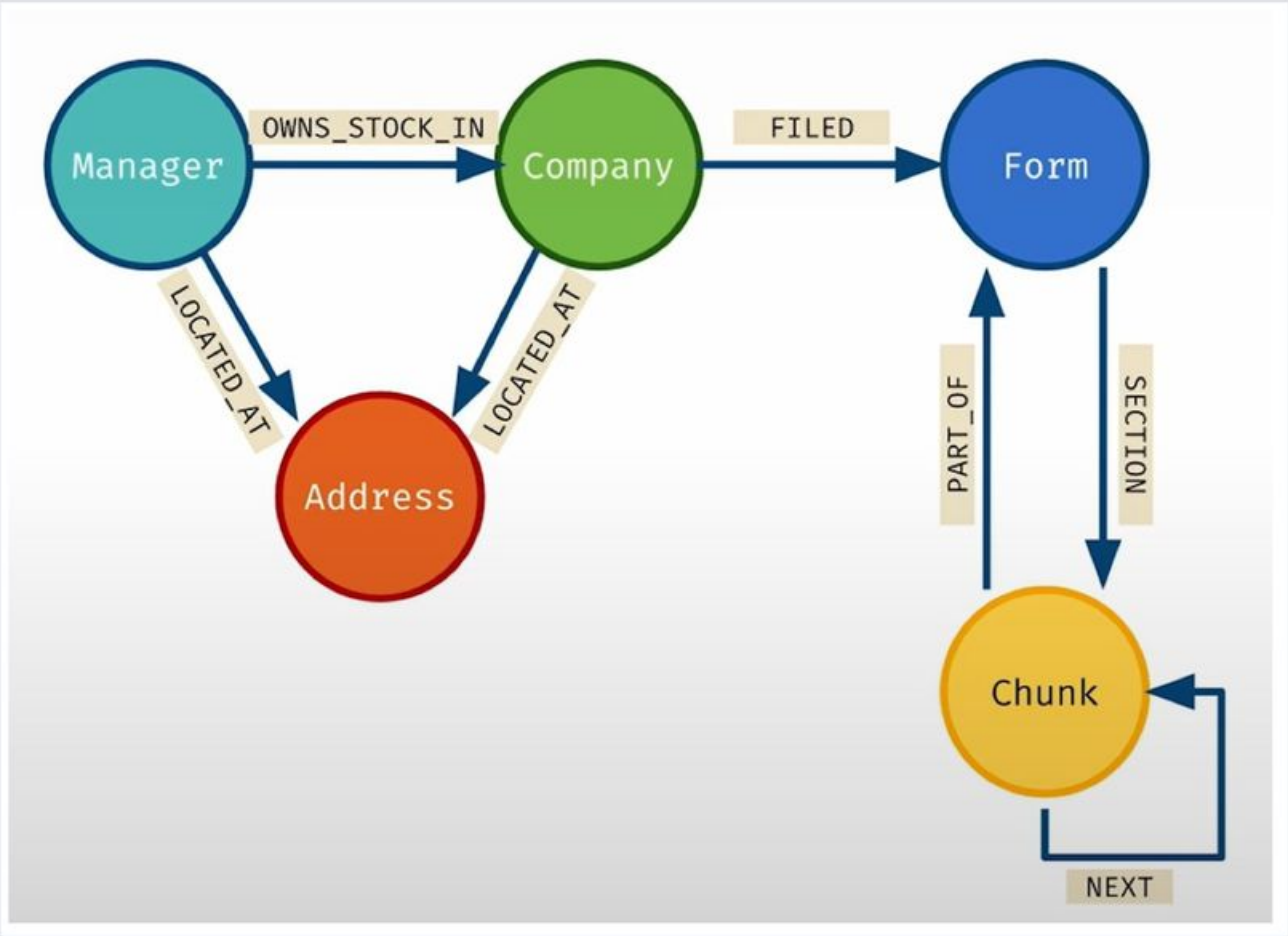
Text-to-Cypher: think like the expert

1. **Parse** — Entities, relationships and constraints in the question
2. **Map** — Onto schema labels and relationship types
3. **Annotate the schema** — Skip list, `/* comments */` on encodings, units, direction
4. **Order the output** — JSON with `relationships` → `reasoning` → `query`
5. **Validate** — `EXPLAIN` before execution; check labels against the schema

> **Listing relationships first reduces invented relationship types.**

Source: notes/11-kg-rag-and-text-to-cypher.md

The failure to expect, and its one-example fix

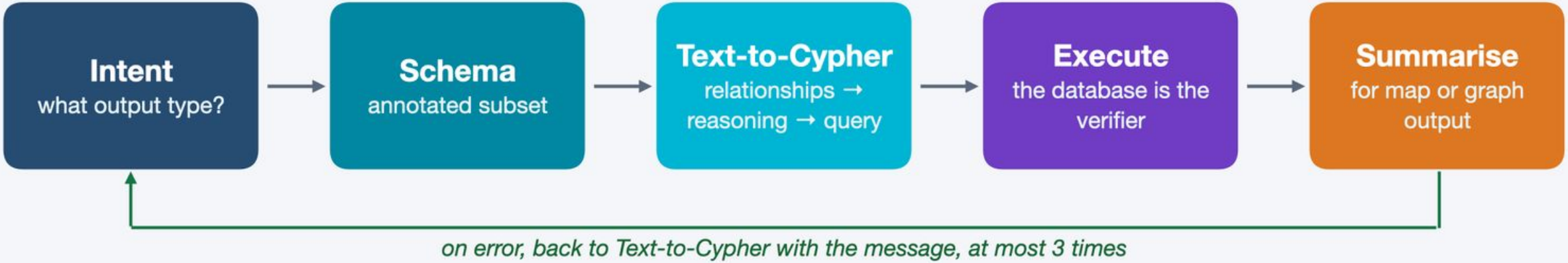


Knowledge Graphs for RAG, DeepLearning.AI × Neo4j (Kollegger, 2024)

- `value` lives on `OWNS_STOCK_IN`, not on `Company`
- The model guesses wrong without help
- **One few-shot example** fixes it
- `GraphCypherQAChain` needs `allow_dangerous_requests=True`

Source: notes/11-kg-rag-and-text-to-cypher.md

The QA agent as a LangGraph graph



- Generate-then-verify with a **bounded back-edge**
- Typed state is the audit trail, and a free dataset of failures
- Missing: an **`EXPLAIN` validator** before execution

Source: notes/11-kg-rag-and-text-to-cypher.md

PART 4

Where the graphs meet

KG as shared agent memory · Learning on graphs

Execution state vs. a knowledge graph

Loop → swarm → graph

All the safety lives in the write path

Three non-negotiables

Build path for a shared-memory swarm

Hand-written features can beat embeddings

GNNs in practice, and predictions as claims

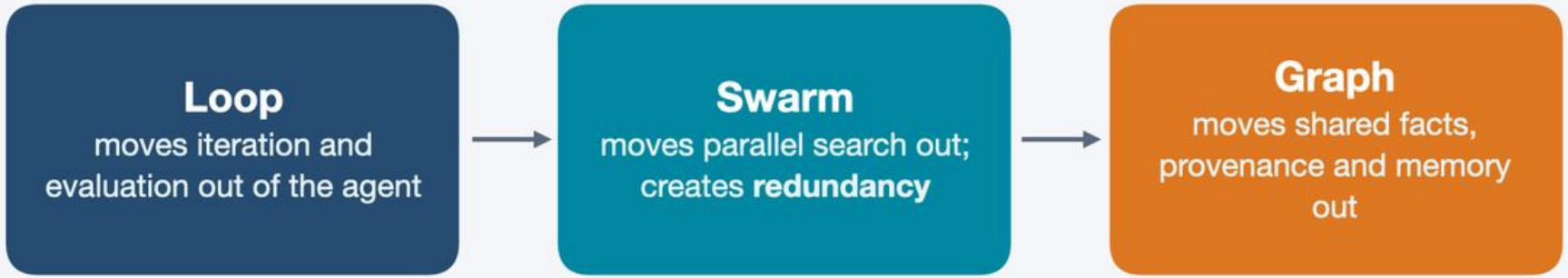
Execution state vs. a knowledge graph

	Execution-graph state	Knowledge graph
Lifetime	One run (`thread_id`)	Across runs and sessions
Readers and writers	Nodes in this graph	Any agent, any run, people too
Shape	Typed dict with reducers	Entities, claims, relations, provenance
Concurrency	Reducers; one writer per key	Validated versioned upserts
Mistakes	Die with the run	**Persist and spread**

> **The agent forgets; the graph does not. That cuts both ways.**

Source: notes/12-kg-as-shared-agent-memory.md

Loop → swarm → graph



- Loop: Karpathy's autoresearch: keep a change only if val bits-per-byte improves
- A swarm alone **re-derives** findings
- A swarm writing to a shared graph **accumulates** them

Source: notes/12-kg-as-shared-agent-memory.md

All the safety lives in the write path

PYTHON

```python
@dataclass
class GraphUpdate:
    nodes: list[dict]   # id, type, props, spans
    edges: list[dict]   # src, type, dst, conf
    run_id: str
    agent_id: str

def publish(update, graph, validator):
    validator.check_schema(update.nodes, update.edges)
    validator.check_provenance(update.nodes, update.edges)
    with graph.transaction() as tx:
        tx.upsert_versioned_nodes(update.nodes)
        tx.add_edges(update.edges)
        tx.link_run(update.run_id, update.agent_id, ...)
        tx.commit()
```

- Workers **submit**, never write directly
- Note 03's reducer at database scale
- Contract: the `GraphUpdate` JSON Schema in note 12

Source: notes/12-kg-as-shared-agent-memory.md

Three non-negotiables

Complete provenance

Every node and edge records `run_id`, `agent_id` and a source span

Reversible resolution

`ALIAS_OF` / `RESOLVES_TO` with evidence; never merge destructively

Claims are not truths

"Agent A claims X, citing S". Contradictions live side by side

Whoever owns the schema owns the errors.

Source: notes/12-kg-as-shared-agent-memory.md

Build path for a shared-memory swarm

1. **One measured loop** — A fixed metric and reversible actions
2. **A role split** — Generator and verifier with isolated contexts
3. **A persistent graph** — Gate: do queries across sessions work?
4. **A swarm writing to it** — Gate: wall-clock gain with **no loss of quality**

> **Evaluate by layer: extraction F1, false-merge rate, path validity, redundant findings.**

Source: notes/12-kg-as-shared-agent-memory.md

Hand-written features can beat embeddings

Graph nodes aren't i.i.d.; connected nodes tend to be alike (homophily).

Features	F1
Degree only	0.43
Node2Vec, 64-dim, untuned	0.65, unstable between runs
Degree + neighbours in faction A + in faction B	**0.93**, stable

> **Features are the bottleneck. Encode the pattern you know is there.**

GNNs in practice, and predictions as claims

PyTorch Geometric (ch12)

Message passing

Each layer aggregates neighbours; k layers = k-hop view. Watch **over-smoothing**

AML (Elliptic)

SAGE best: caught **82.7%** of illicit nodes; GCN missed **32%**

GNN + LLM

LLM as predictor, as encoder of node text, or as aligner

Predictions are claims

Write via `GraphUpdate` with `confidence`; keep out of default traversals

A false predicted link does what a false merge does. Watch the false-positive rate.

Source: notes/13-graph-ml-and-gnns.md

Rules of thumb

First graph

Generate-then-verify is the highest-yield first graph

Agency

Code where you can, one LLM call where you must, an agent only where the path is unknown

State

Every shared key has one writer or a reducer

Gates

Gate the edges where consequences concentrate, not every node

Verifiers

See the task and the output, never the generator's reasoning

Anchors

Keep at least one anchor the graph can't edit

Source: README.md#rules-of-thumb

Where to go next

- **Part A labs (A1–A7):** grow one graph through Parts 1–2 of this talk
- **Part B labs (B1–B9):** the Neo4j course, your own domain, a shared-memory swarm
- **The notes:** every slide here has its code and figures in notes 01–13
- **Knowledge Graphs for RAG:** the hosted DeepLearning.AI × Neo4j course
- **06-eval:** measure whether the whole system is correct

The one idea to keep

Start with a loop. Add a node only when a specific signal forces it.

Source: README.md